

UNIVERSITÀ DEGLI STUDI DI
NAPOLI FEDERICO II



CORSO DI LAUREA MAGISTRALE IN INGEGNERIA
INFORMATICA

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE
TECNOLOGIE DELL'INFORMAZIONE

Safety Critical Systems
Gestione di pazienti in terapia intensiva

Candidati:

Vincenzo D'Angelo M63001595

Giorgio Di Costanzo M63001579

Professoressa:

Valeria Vittorini

Anno Accademico 2024/2025

Indice

1	Introduzione	1
2	Esercizio GSPN	2
2.1	Descrizione del sistema reale	2
2.2	Traduzione in rete GSPN	2
2.3	Parametri temporali	6
2.4	Analisi delle proprietà strutturali e comportamentali	7
2.4.1	Boundedness	7
2.4.2	Liveness	7
2.4.3	Reachability	8
2.4.4	Reversibility	8
2.5	Calcolo degli indici prestazionali	10
2.5.1	Probabilità di essere in uno stato critico	10
2.5.2	Indicatori sintetici di performance	10
2.6	Analisi quantitativa ed economica	12
2.6.1	Indicatori prestazionali per la configurazione 1 (5 letti, 1 medico, 3 infermieri)	12
2.6.2	Indicatori prestazionali per la configurazione 2 (10 letti, 3 medici, 5 infermieri)	13
2.6.3	Considerazioni finali	14
3	Model Checking	15
3.1	Modello centralizzato	16
3.1.1	Verifica delle proprietà formali	19
3.2	Modello multiprocesso	21
3.2.1	Modulo main	21
3.2.2	Verifica delle proprietà formali	24

1 Introduzione

La gestione dei pazienti in terapia intensiva rappresenta una sfida impegnativa per il sistema sanitario, poiché richiede un utilizzo efficiente di risorse critiche come medici, infermieri, attrezzature specialistiche e posti letto. In questo contesto, risulta fondamentale poter modellare e analizzare il comportamento del sistema al variare di condizioni operative, tassi di guasto e disponibilità del personale, al fine di garantire livelli adeguati di qualità, sicurezza e continuità delle cure.

Le Reti di Petri Stocastiche Generalizzate (GSPN) offrono un potente strumento di modellazione per sistemi caratterizzati da concorrenza, sincronizzazione e comportamenti stocastici. In particolare, consentono di rappresentare in modo naturale eventi asincroni, competizione tra risorse, tempi di attesa e scenari di guasto e ripristino.

L'obiettivo di questo lavoro è quello di costruire un modello GSPN per la simulazione del processo di gestione dei pazienti in un reparto di terapia intensiva, con particolare attenzione alla disponibilità del personale sanitario e alle dinamiche legate all'accesso alle cure in condizioni di criticità. Il modello è stato analizzato mediante la verifica delle proprietà comportamentali (boundedness, liveness, reversibility, ecc.) e successivamente validato tramite model checking, utilizzando logica temporale per formalizzare e verificare scenari di interesse critico. Inoltre, è stato condotto il calcolo di indici prestazionali rilevanti, come la probabilità di trovarsi in uno stato critico e il throughput di transizioni chiave, al fine di valutare le prestazioni e la resilienza del sistema simulato.

Infine, viene proposta un'analisi quantitativa che esplora l'impatto di diversi scenari sul comportamento del sistema, includendo valutazioni economiche legate al potenziamento delle risorse e al rafforzamento delle capacità operative.

Per la costruzione e la simulazione del modello di rete di Petri è stato utilizzato il tool PIPE [1] (Platform Independent Petri net Editor) versione 4.3.0, che consente di definire graficamente la rete e di eseguire simulazioni stocastiche.

Per la verifica formale delle proprietà del modello, come l'analisi di correttezza e la validazione tramite model checking, è stato impiegato il tool NuSMV [2] versione 2.5.4, un model checker simbolico che supporta la verifica di sistemi concorrenti e stocastici.

2 Esercizio GSPN

2.1 Descrizione del sistema reale

In un'unità di terapia intensiva (ICU), i pazienti ricoverati vengono monitorati costantemente. In presenza di anomalie nei parametri vitali, viene generato un allarme che richiede l'intervento tempestivo del personale sanitario. La gestione dei pazienti segue una logica di priorità: se un medico è disponibile, interviene direttamente; altrimenti, un infermiere può occuparsi del paziente solo dopo aver ricevuto istruzioni dal medico.

Il modello proposto si basa su alcune assunzioni semplificative e vincoli operativi, in particolare:

- L'unità dispone di 3 letti.
- Ogni letto può essere occupato da un solo paziente alla volta.
- Il numero di risorse sanitarie è limitato: si assume la disponibilità di un medico e due infermieri.
- Gli infermieri intervengono solo dopo consulto con un medico.
- I pazienti possono attendere in coda in assenza di risorse, con una coda massima ammessa pari a 20 pazienti.

Per la costruzione del modello è stato adottato un approccio top-down, partendo dall'analisi generale del sistema reale e procedendo con la sua scomposizione in componenti più dettagliate fino alla definizione completa della rete.

2.2 Traduzione in rete GSPN

Il sistema è modellato tramite una rete di Petri stocastica generalizzata (GSPN), dove i pazienti sono rappresentati da token, gli stati del sistema corrispondono ai posti e le transizioni rappresentano eventi stocastici.

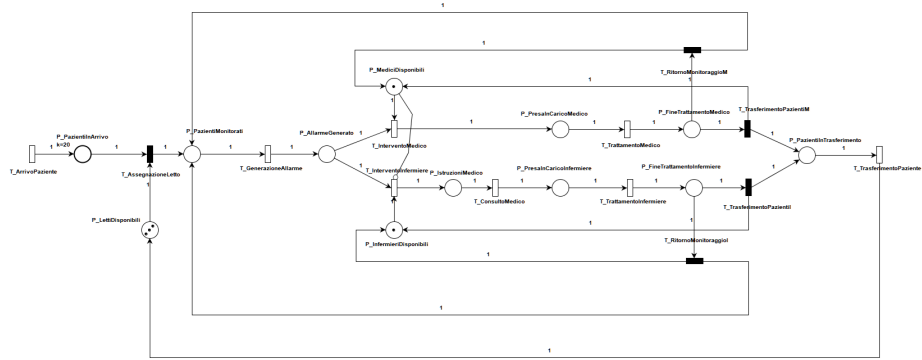


Figura 1: Petri Net

Descrizione dei posti

Posto	Significato
P_PazientiInArrivo	Pazienti in attesa di essere assegnati a un letto.
P_LettiDisponibili	Letti disponibili nell'unità. Token = numero di letti liberi.
P_PazientiMonitorati	Pazienti sotto monitoraggio in terapia intensiva.
P_AllarmeGenerato	Stato in cui un allarme è attivo per un paziente.
P_MediciDisponibili	Medici disponibili per il trattamento.
P_InfermieriDisponibili	Infermieri disponibili per il trattamento.
P_PresaInCaricoMedico	Il paziente è stato preso in carico da un medico.
P_IstruzioniMedico	Il medico ha fornito le istruzioni necessarie per l'intervento dell'infermiere.
P_PresaInCaricoInfermiere	Il paziente è stato preso in carico da un infermiere.
P_FineTrattamentoMedico	Il trattamento da parte del medico è terminato.
P_FineTrattamentoInfermiere	Il trattamento da parte dell'infermiere è terminato.
P_PazientiInTrasferimento	Il paziente è in fase di uscita dall'unità, per dimissione o trasferimento in altro reparto.

Tabella 1: Significato dei posti nel modello GSPN

Per garantire che la rete di Petri sia limitata e rappresenti in modo realistico il sistema, il posto P_PazientiInArrivo è stato vincolato con una capacità massima di 20 token. Questo limite riflette la capacità massima di pazienti

in attesa di ingresso nel sistema e permette di evitare l'accumulo illimitato di token, mantenendo così la rete analizzabile e coerente con la realtà operativa dell'unità di terapia intensiva. Il valore 20 è stato scelto empiricamente: valori inferiori tendevano a saturare frequentemente il posto, impedendo l'accettazione di nuovi pazienti.

Per rappresentare in modo dettagliato la fase di trattamento, sono stati introdotti due posti distinti: uno per il trattamento effettuato dal medico e uno per quello eseguito dall'infermiere. Questa distinzione si è resa necessaria per poter tracciare correttamente chi interviene su ciascun paziente e calcolare indicatori specifici, come la probabilità che il trattamento venga effettuato da un medico.

L'arco inibitore che collega il posto `P_MediciDisponibili` alla transizione `T_InterventoInfermiere` ha lo scopo di modellare correttamente la logica di priorità nell'intervento sanitario. In particolare, secondo le regole operative del sistema, un infermiere può intervenire solo nel caso in cui nessun medico sia disponibile.

L'arco inibitore implementa quindi una condizione di abilitazione negativa: la transizione `T_InterventoInfermiere` risulta abilitata solo quando il posto `P_MediciDisponibili` è vuoto, ovvero quando il numero di medici disponibili è pari a zero.

Descrizione delle transizioni

Transizione	Descrizione
T.ArrivoPaziente	Transizione temporizzata che modella l'arrivo di un nuovo paziente nel sistema.
T.AssegnazioneLetto	Transizione temporizzata che rappresenta l'assegnazione di un letto disponibile a un paziente.
T.GenerazioneAllarme	Transizione temporizzata che simula la generazione di un allarme a seguito di segni critici del paziente.
T.InterventoMedico	Transizione temporizzata che rappresenta l'intervento di un medico per prendere in carico il paziente.
T.InterventoInfermiere	Transizione temporizzata che rappresenta l'intervento di un infermiere per prendere in carico il paziente.
T.ConsultoMedico	Transizione temporizzata che modella il medico mentre fornisce istruzioni a un infermiere.
T.TrattamentoMedico	Transizione temporizzata che rappresenta la durata del trattamento effettuato da un medico.
T.TrattamentoInfermiere	Transizione temporizzata che rappresenta la durata del trattamento effettuato da un infermiere.
T.RitornoMonitoraggioM	Transizione immediata che modella il ritorno del paziente alla fase di monitoraggio dopo la fine del trattamento medico.
T.RitornoMonitoraggioI	Transizione immediata che modella il ritorno del paziente alla fase di monitoraggio dopo la fine del trattamento infermieristico.
T.TrasferimentoPazientiM	Transizione immediata che sposta il paziente nell'area di uscita o trasferimento a seguito del trattamento medico.
T.TrasferimentoPazientiI	Transizione immediata che sposta il paziente nell'area di uscita o trasferimento a seguito del trattamento infermieristico.
T.TrasferimentoPaziente	Transizione temporizzata che modella la dimissione o il trasferimento del paziente in un'altra unità.

Tabella 2: Significato delle transizioni nel modello GSPN

La transizione T_ArrivoPaziente è una *source transition*, in quanto non è vincolata da alcun posto in ingresso: rappresenta l'ingresso di nuovi pazienti nel sistema, generando token in modo autonomo. Analogamente, la transizione T_TrasferimentoPaziente rappresenta l'uscita definitiva del paziente

dall'unità (dimissione o trasferimento), rimuovendo il token dal sistema senza generare ulteriori evoluzioni nella rete. Non è tuttavia classificabile come *sink transition*, poiché restituisce il token al posto `P_LettiDisponibili`. In alternativa, si potrebbe modellare il trasferimento come *sink transition* rilasciando la risorsa nella transizione precedente, ma in tal caso sarebbe necessario limitare la capacità del posto `P_PazientiInTrasferimento` per evitare un accumulo non controllato di token.

Le risorse relative a medici e infermieri vengono liberate sulle transizioni `T_RitornoMonitoraggio`, `T_TrasferimentoPazientiI` e `T_TrasferimentoPazientiM`, a indicare che tali operatori non sono coinvolti nella fase di trasferimento del paziente.

La rete modellata presenta un totale di 12 posti e 13 transizioni.

2.3 Parametri temporali

Per la costruzione del modello è stata definita una serie di parametri temporali rappresentativi delle diverse fasi e processi coinvolti nella gestione dei pazienti in terapia intensiva. I valori medi, espressi in minuti, riflettono i tempi tipici osservati o stimati per ciascun evento chiave nel sistema, e sono utilizzati per la parametrizzazione delle transizioni stocastiche nella rete di Petri.

Evento o processo	Tempo medio (min)	λ (1/min)
Arrivo di un paziente	10.0	0.100
Tempo di generazione di un allarme	2.0	0.500
Tempo di risposta del medico	1.0	1.000
Tempo di risposta dell'infermiere	1.5	0.667
Tempo di istruzione medico $\rightarrow$ infermiere	0.5	2.000
Durata del trattamento	2.0	0.500
Tempo medio prima dell'uscita dall'unità	4.0	0.250

Tabella 3: Tempi medi e tassi associati alle transizioni stocastiche

Il tempo medio di arrivo di un paziente è stato aumentato da 5 minuti (come indicato nella specifica iniziale) a 10 minuti. Questa modifica mantiene il modello coerente con scenari di emergenza, ma consente una gestione più equilibrata dei pazienti, garantendo una dinamica sostenibile tra ingressi e dimissioni e migliorando l'osservabilità delle politiche di trattamento.

Questi parametri costituiscono la base per la simulazione e l'analisi degli indici prestazionali, che verranno approfonditi in seguito.

2.4 Analisi delle proprietà strutturali e comportamentali

2.4.1 Boundedness

Una rete di Petri è detta *bounded* se esiste un numero intero positivo k (finito) tale che, in ogni marcatura raggiungibile dalla marcatura iniziale m_0 , il numero di token presenti in ciascun posto della rete sia sempre minore o uguale a k . In altre parole, nessun posto può contenere un numero illimitato di token durante l'evoluzione del sistema.

Una rete di Petri è detta *safe* se è 1-bounded, ovvero se in ogni marcatura raggiungibile ogni posto contiene al più un token.

Nel modello sviluppato, l'analisi condotta con PIPE ha evidenziato l'assenza di P-invarianti positive che coprano l'intera rete. Questo risultato implica che non è possibile identificare una combinazione lineare di posti (con coefficienti strettamente positivi) tale per cui il numero totale di token nel sistema rimanga costante per ogni marcatura raggiungibile.

Le violazioni del principio di conservazione risiedono nelle transizioni che generano un numero di token diverso da quello consumato (come `T_AssegnazioneLetto`), nella source transition `T_ArrivoPaziente`, che introduce nuovi token, e nella sink transition `T_TrasferimentoPaziente`, che li rimuove definitivamente dal sistema.

Tuttavia, la presenza di limiti espliciti sulle capacità dei posti assicura che la rete sia comunque limitata dal punto di vista comportamentale. Questo rende l'assenza di P-invarianti strutturali coerente con la natura dinamica e aperta del sistema reale rappresentato.

In conclusione, la rete risulta essere limitata, ma non safe, in quanto alcuni posti possono contenere più di un token nelle marcature raggiungibili.

2.4.2 Liveness

Una transizione t è *viva* in un sistema PN se e solo se per ogni marcatura m raggiungibile da m_0 esiste una marcatura m' raggiungibile da m tale che t è abilitata in m' . Un sistema PN è vivo se tutte le sue transizioni sono vive.

Nel modello analizzato, la rete è coperta da T-invarianti positive, come indicato dall'analisi automatica condotta tramite PIPE. Questo suggerisce che esistono cicli interni all'interno della rete, attraverso cui è possibile riportare il sistema in una configurazione equivalente a quella di partenza.

Il fatto che tutte le transizioni siano contenute almeno in una T-invariante implica che, dal punto di vista strutturale, ogni transizione può far parte di un comportamento ciclico potenzialmente ripetibile, il che è un buon indicatore per garantire la liveness a livello strutturale.

Tuttavia, la presenza di T-invarianti non è sufficiente da sola a garantire la liveness comportamentale, che va verificata considerando la marcatura iniziale

e i vincoli imposti alle risorse.

Il comportamento osservato durante le simulazioni del modello suggerisce che tutte le transizioni siano potenzialmente abilitabili, a condizione che le risorse vengano opportunamente liberate e i vincoli di capacità rispettati. In particolare:

- La transizione `TArrivoPaziente` è sempre abilitata fintanto che la coda non è piena;
- Le transizioni di intervento sono abilitabili a seconda della disponibilità di risorse;
- Le transizioni di ritorno al monitoraggio e trasferimento permettono di liberare risorse, evitando deadlock permanenti.

Pertanto, la liveness del sistema è garantita a patto di una corretta gestione delle risorse.

2.4.3 Reachability

La proprietà di raggiungibilità consiste nella capacità del sistema di evolvere da una marcatura iniziale m_0 ad una qualsiasi altra marcatura m mediante una sequenza finita di transizioni. L'insieme di tutte le marcature raggiungibili definisce il *Reachability Set* (RS), la cui esplorazione consente di valutare la correttezza e la completezza del modello.

Una conseguenza molto importante della boundedness è che essa implica la finitezza dello spazio degli stati. In particolare, si può dimostrare che se un modello di rete di Petri costituito da N posti è k -bounded, allora il numero di marcature nel suo RS non può superare $(k + 1)^N$.

Nel caso del modello proposto, con $N = 12$ posti e supponendo $k = 20$ come valore massimo per la capacità del posto dei pazienti in arrivo, la cardinalità massima teorica dello spazio degli stati è pari a $(20 + 1)^{12}$ marcature possibili. Sebbene questo garantisca che il sistema sia finito, il numero di stati è così elevato da rendere impraticabile la generazione completa del grafo di raggiungibilità tramite strumenti come PIPE. In particolare, la costruzione del *Reachability Graph* e anche del solo *Reachability Set* risulta inattuabile per via dell'elevato costo computazionale legato all'esplorazione esaustiva dello spazio delle marcature.

In tali casi, si rende necessario ricorrere ad approcci alternativi come simulazioni parziali o tecniche simboliche di model checking.

2.4.4 Reversibility

Una rete di Petri è detta *reversibile* se, a partire da qualsiasi marcatura m raggiungibile dalla marcatura iniziale m_0 , è possibile ritornare a m_0 .

La reversibilità garantisce che il sistema possa sempre ritornare al suo stato iniziale, indipendentemente dall'evoluzione del processo, ed è particolarmente rilevante per sistemi ciclici o che devono mantenere una certa stabilità nel tempo.

Una condizione più debole della reversibilità è l'esistenza di uno *home state*, cioè una marcatura m' tale che da ogni marcatura raggiungibile m , sia possibile raggiungere m' .

Nel caso in esame, la reversibilità completa del sistema è garantita solo in condizioni particolari, ovvero in assenza di nuovi arrivi di pazienti. In tale scenario, il sistema può svuotarsi progressivamente fino a tornare allo stato iniziale.

Tuttavia, nella pratica, data la continua generazione di nuovi arrivi e la presenza di risorse limitate, si osserva che il sistema tende a raggiungere e permanere in uno o più *home state*, ovvero marcature che risultano frequentemente visitate e raggiungibili da ogni altro stato. Un esempio tipico di *home state* è rappresentato da una situazione in cui il reparto è prossimo alla massima capacità operativa, con tutti i letti occupati e pazienti in attesa.

2.5 Calcolo degli indici prestazionali

In questa sezione vengono presentati gli indicatori prestazionali utilizzati per valutare il comportamento del sistema modellato tramite GSPN. Gli indici permettono di analizzare la qualità del servizio, l'efficienza nella gestione delle risorse e la probabilità di condizioni critiche.

2.5.1 Probabilità di essere in uno stato critico

Uno *stato critico* è definito come una marcatura in cui un paziente necessita di trattamento urgente ma nessuna risorsa è disponibile per intervenire (né medico, né infermiere autorizzato). La probabilità di essere in uno stato critico si ottiene come:

$$\mathbb{P}(\text{evento critico}) = \sum_{m \in H} \pi(m) = 0.038350 \approx 3.84\%$$

dove H è l'insieme degli stati identificati come critici e $\pi(m)$ è la probabilità stazionaria associata allo stato m .

L'identificazione degli stati critici è stata effettuata in base alla configurazione della rete, considerando le marcature in cui è presente almeno un token nel posto `P_AllarmeGenerato`, mentre i posti `P_MediciDisponibili` e `P_InfermieriDisponibili` risultano entrambi vuoti.

Poiché PIPE non consente di definire direttamente tali condizioni sulle marcature, il calcolo della probabilità complessiva di evento critico è stato effettuato tramite uno script Python che elabora il file CSV delle marcature e somma le probabilità stazionarie associate agli stati che soddisfano la condizione descritta.

2.5.2 Indicatori sintetici di performance

A partire dalla simulazione o dall'analisi stazionaria, si possono ricavare i seguenti indici di performance globali del sistema.

Tasso medio di occupazione dei letti

Il numero medio di letti occupati è dato dalla differenza tra la capacità totale e il valore atteso dei token nel posto `P_LettiDisponibili`:

$$\begin{aligned} \text{Letti mediamente occupati} &= N_{\text{letti}} - \mathbb{E}[\text{Token in P_LettiDisponibili}] = \\ &= 3 - 0.62821 = 2.37179 \end{aligned}$$

Dividendo questo valore per il numero totale di letti, si ottiene il tasso medio di occupazione, che rappresenta la frazione di letti occupati nel tempo:

$$\begin{aligned} \text{Tasso di occupazione letti} &= \frac{N_{\text{letti}} - \mathbb{E}[\text{Token in P_LettiDisponibili}]}{N_{\text{letti}}} = \\ &= \frac{2.37179}{3} = 0.79059 \approx 79.05\% \end{aligned}$$

Numero medio di pazienti in attesa o in trattamento

Somma dei token medi nei posti che rappresentano pazienti in attesa o sottoposti a trattamento:

$$\begin{aligned}\mathbb{E}[P_{\text{in_attesa}} \text{ o } P_{\text{in_trattamento}}] &= \mathbb{E}[P_{\text{AllarmeGenerato}}] \\ &\quad + \mathbb{E}[P_{\text{PresaInCaricoMedico}}] \\ &\quad + \mathbb{E}[P_{\text{PresaInCaricoInfermiere}}] = \\ &= 0.42586 + 0.39743 + 0.13473 = 0.95802\end{aligned}$$

Probabilità che il trattamento sia effettuato da un medico

La probabilità che un trattamento venga effettuato da un medico, piuttosto che da un infermiere, può essere stimata utilizzando i throughput medi delle rispettive transizioni di trattamento. Indicando con:

$$\text{Throughput}(\text{T_TrattamentoMedico}) = \mu_M$$

$$\text{Throughput}(\text{T_TrattamentoInfermiere}) = \mu_I$$

la frazione di trattamenti complessivi eseguiti da un medico sul totale dei trattamenti erogati risulta pari a:

$$\mathbb{P}(\text{Medico}) = \frac{\mu_M}{\mu_M + \mu_I} = \frac{0.19872}{0.19872 + 0.06737} = 0.74681 \approx 74.68\%$$

Throughput complessivo del sistema

Il throughput complessivo del sistema rappresenta la frequenza media con cui i pazienti vengono trattati, indipendentemente dal tipo di operatore. Esso si ottiene come somma dei throughput medi delle due transizioni di trattamento (medico e infermiere):

$$\text{Throughput} = \mu_M + \mu_I = 0.19872 + 0.06737 = 0.26609$$

Questo valore corrisponde, in media, a circa un paziente trattato ogni 3.76 minuti.

Questi indicatori forniscono una base quantitativa per valutare l'efficacia delle risorse, l'adeguatezza della configurazione del sistema e per supportare eventuali strategie di miglioramento.

2.6 Analisi quantitativa ed economica

In questo capitolo si analizza l'impatto di diversi scenari sul comportamento complessivo del sistema, con particolare attenzione agli effetti derivanti dal potenziamento delle risorse e dal rafforzamento delle capacità operative, includendo anche considerazioni di tipo economico. Nel dettaglio, mantenendo costanti il tasso di arrivo (un paziente ogni dieci minuti) e il limite massimo di pazienti in coda (20), si è scelto di variare il numero di posti letto, medici e infermieri. In particolare, sono state considerate le seguenti configurazioni:

- Configurazione 1:
 - Posti letto = 5
 - Medici = 1
 - Infermieri = 3
- Configurazione 2:
 - Posti letto = 10
 - Medici = 3
 - Infermieri = 5

Per ciascuna configurazione sono stati calcolati gli indicatori prestazionali definiti nel paragrafo precedente.

2.6.1 Indicatori prestazionali per la configurazione 1 (5 letti, 1 medico, 3 infermieri)

L'analisi della prima configurazione ha prodotto i seguenti risultati prestazionali:

- Probabilità di *evento critico*:

$$\mathbb{P}(\text{evento critico}) = \sum_{m \in H} \pi(m) = 0.00166 \approx 0.17\%$$

- Tasso medio di utilizzo dei letti:

$$\text{Occupazione}_{\text{letti}} = \frac{2.60689}{5} = 0.521378 \approx 52.13\%$$

- Probabilità che il trattamento sia effettuato da un medico:

$$\mathbb{P}(\text{Medico}) = \frac{0,18524}{0,18524 + 0,07651} = 0,70769 \approx 70.77\%$$

- Throughput complessivo del sistema:

$$\mu_M + \mu_I = 0.18524 + 0.07651 = 0.26175 \text{ (circa una persona ogni 3.82 minuti)}$$

Rispetto alla configurazione iniziale, è possibile osservare una diminuzione della probabilità di evento critico, coerente con l'aumento delle risorse disponibili. Come previsto, il tasso di occupazione dei letti diminuisce (in seguito all'aumento del numero di posti letto a 5), così come si riduce la probabilità che il trattamento venga effettuato da un medico, dato che in questa configurazione il medico è unico mentre gli infermieri sono tre.

Infine, il throughput complessivo diminuisce, in linea con il fatto che l'intervento dell'infermiere richiede più tempo rispetto a quello del medico.

2.6.2 Indicatori prestazionali per la configurazione 2 (10 letti, 3 medici, 5 infermieri)

L'analisi della seconda configurazione ha prodotto i seguenti risultati prestazionali:

- Probabilità di *evento critico*:

$$\mathbb{P}(\text{evento critico}) = \sum_{m \in H} \pi(m) = 0.00001 \approx 0.001\%$$

- Tasso medio di utilizzo dei letti:

$$\text{Occupazione}_{\text{letti}} = \frac{3.38159}{10} = 0.338159 \approx 33.82\%$$

- Probabilità che il trattamento sia effettuato da un medico:

$$\mathbb{P}(\text{Medico}) = \frac{0.26801}{0.26801 + 0.02133} = 0,92628 \approx 92.62\%$$

- Throughput complessivo del sistema:

$$\mu_M + \mu_I = 0.26801 + 0.02133 = 0.28934 \text{ (circa una persona ogni 3.47 minuti)}$$

La seconda configurazione, caratterizzata da un incremento significativo delle risorse disponibili, mostra un netto miglioramento negli indicatori prestazionali rispetto alla configurazione precedente.

In particolare, la probabilità di un evento critico si riduce drasticamente e il tasso medio di utilizzo dei letti scende al 33.82%, indicativo di una minore saturazione del reparto.

La probabilità che il trattamento venga effettuato da un medico raggiunge il 92.62%. Questo dato riflette la presenza di un numero più elevato di medici rispetto agli infermieri, portando a un aumento significativo dell'intervento medico diretto, che garantisce trattamenti più rapidi ed efficaci.

Infine, il throughput complessivo del sistema migliora leggermente, passando a circa una persona trattata ogni 3.47 minuti.

Nel complesso, questa configurazione si dimostra più efficiente e sicura, ma a fronte di un maggior impiego di risorse, aspetto da considerare nelle valutazioni economiche e organizzative.

2.6.3 Considerazioni finali

L'ultima configurazione è senza dubbio la migliore in termini di indici prestazionali, poichè dimostra miglioramenti significativi nella gestione dei pazienti e nella riduzione degli eventi critici. Tuttavia, essa comporta un costo molto maggiore rispetto alle configurazioni precedenti, dovuto all'aumento consistente di letti, medici e infermieri.

La seconda configurazione, invece, rappresenta un buon compromesso tra costi e performance, garantendo un livello accettabile di sicurezza e efficienza con un impiego di risorse più contenuto.

Un corretto dimensionamento del sistema dovrebbe quindi essere guidato da specifiche chiare riguardanti la soglia minima accettabile di eventi critici e dalla necessità di mantenere un numero minimo di letti disponibili nel reparto, al fine di bilanciare adeguatamente sicurezza, qualità del servizio e sostenibilità economica.

3 Model Checking

Il model checking è un metodo automatico e rigoroso che permette di analizzare tutte le possibili evoluzioni temporali del sistema, garantendo la correttezza di proprietà critiche come la sicurezza, la disponibilità delle risorse e la gestione degli allarmi.

Attraverso l'uso di logiche temporali, quali CTL e LTL, si definiscono specifiche formali che descrivono il comportamento desiderato del sistema. Successivamente, grazie all'impiego del tool NuSMV, queste proprietà vengono verificate, fornendo una solida conferma della validità del modello e permettendo di individuare eventuali anomalie o situazioni non desiderate.

Il model checking rappresenta così un complemento essenziale alle tecniche di simulazione e analisi strutturale utilizzate in precedenza, rafforzando la validazione complessiva del modello e supportando la progettazione di sistemi più affidabili e sicuri.

L'obiettivo dell'esercizio è verificare, tramite tecniche di model checking, la correttezza logica del comportamento del sistema e la sua capacità di risposta in scenari ordinari e di emergenza. La gestione del trattamento avviene secondo la seguente logica:

- Se un medico è disponibile, interviene direttamente in risposta all'allarme.
- Se il medico non è disponibile, può intervenire un infermiere, ma solo dopo aver ricevuto istruzioni dal medico.
- Il trattamento, se attivato, interrompe la situazione di allarme.
- Se l'allarme persiste per tre intervalli di tempo consecutivi senza che venga attivato un trattamento, si verifica un evento critico, indicativo di un mancato intervento tempestivo.

La verifica formale è stata condotta su due varianti del modello:

1. Modello centralizzato (`centralizzato.smv`): rappresenta il sistema come un insieme di variabili globali, senza struttura a processi. La concorrenza è gestita in modo implicito, modellando operatori impegnati come semplicemente "occupati".
2. Modello multiprocesso (`multiprocesso.smv`): adotta più processi espliciti per rappresentare pazienti concorrenti, permettendo una modellazione più fedele di situazioni simultanee e conflitti di risorse.

3.1 Modello centralizzato

Il modulo `main` definisce le variabili di stato che modellano la disponibilità degli operatori (medico e infermiere), lo stato del paziente, la presenza di un evento critico, e un contatore che misura la durata dell'allarme senza trattamento.

```
VAR
medico : {disponibile, occupato, trattamento};
infermiere : {disponibile, occupato, istruzioni, trattamento
};
paziente : {osservazione, allarme, trattamento};
evento_critico : boolean;
tempo_allarme : 0..3;
```

Codice 1: Variabili di stato modello centralizzato

Il valore `occupato` per le variabili `medico` e `infermiere` rappresenta una condizione in cui l'operatore è impegnato in attività esterne alla gestione del paziente, come ad esempio il trattamento di altri pazienti o compiti clinici generici. Lo stato `trattamento`, invece, indica che l'operatore è attivamente coinvolto nel trattamento del paziente in esame, a seguito di un allarme. Si distingue infine lo stato `istruzioni`, valido solo per l'infermiere, che rappresenta una fase preparatoria in cui quest'ultimo riceve indicazioni dal medico prima di procedere con il trattamento.

Si passa quindi alla definizione dello stato iniziale del sistema: entrambi gli operatori sono disponibili, il paziente è in osservazione, non è presente alcun evento critico e il contatore dell'allarme è azzerato.

```
INIT
medico = disponibile &
infermiere = disponibile &
paziente = osservazione &
evento_critico = FALSE &
tempo_allarme = 0;
```

Codice 2: Stato iniziale modello centralizzato

Successivamente, vengono definite le regole di transizione per ciascuna variabile di stato:

ASSIGN

```
-- Evoluzione dello stato del paziente
next(paziente) := case
  paziente = osservazione : {osservazione, allarme}; -- l'
    allarme può generarsi
  paziente = allarme & (medico = disponibile | infermiere =
    istruzioni) : trattamento;
  paziente = trattamento : osservazione; -- trattamento
    concluso
  TRUE : paziente;
esac;

-- Evoluzione del tempo dell'allarme
next(tempo_allarme) := case
  paziente = allarme & tempo_allarme < 3 : tempo_allarme + 1;
  paziente != allarme : 0; -- resetta se non è in allarme
  TRUE : tempo_allarme;
esac;

-- Evento critico: scatta se il paziente resta troppo a lungo
  in allarme senza trattamento
next(evento_critico) := case
  tempo_allarme = 2 & medico = occupato & (infermiere =
    disponibile | infermiere = occupato) : TRUE;
  paziente = trattamento : FALSE;
  TRUE : evento_critico;
esac;

-- Evoluzione del medico
next(medico) := case
  paziente = allarme & medico = disponibile & infermiere !=
    istruzioni: trattamento;
    medico = trattamento : disponibile;
    medico = disponibile & paziente != allarme : {
      disponibile, occupato};
    medico = occupato : {disponibile, occupato};
  TRUE : medico;
esac;

-- Evoluzione dell'infermiere
next(infermiere) := case
  paziente = allarme & medico != disponibile & infermiere =
    disponibile : istruzioni;
  infermiere = istruzioni : trattamento;
  infermiere = trattamento : disponibile;
  infermiere = disponibile & (paziente != allarme |
```

```

        paziente = allarme & medico = disponibile) : {
            disponibile, occupato};
    infermiere = occupato : {disponibile, occupato};
    TRUE : infermiere;
esac;

```

Codice 3: Transizioni di stato modello centralizzato

In particolare:

- quando il paziente si trova in osservazione, può rimanere in osservazione o entrare in allarme (scelta non deterministica). Se il paziente è in allarme, può passare a trattamento se il medico è disponibile, oppure l'infermiere ha ricevuto istruzioni. Al termine del trattamento passa di nuovo allo stato di osservazione.
- se il paziente è in allarme, il contatore `tempo_allarme` si incrementa ogni passo fino a un massimo di 3. Quando il paziente non è più in stato di allarme il contatore viene azzerato. In tutti gli altri casi, mantiene il valore corrente.
- un evento critico viene segnalato se il paziente rimane in stato di allarme per tre intervalli consecutivi. In particolare, ciò avviene quando, con `tempo_allarme = 2`, né il medico né l'infermiere sono in grado di intervenire. L'evento critico si disattiva non appena ha inizio un trattamento. Negli altri casi, mantiene lo stato corrente.
- se il paziente è in allarme e il medico è disponibile (e l'infermiere non ha ricevuto istruzioni), allora il medico inizia direttamente il trattamento. Dopo il trattamento, torna disponibile. Se il medico è disponibile e il paziente non è in allarme, può passare a occupato, mentre se è già occupato, può restare tale o tornare disponibile.
- se il paziente è in allarme, il medico non è disponibile e l'infermiere è disponibile, allora riceve istruzioni. Dopo aver ricevuto istruzioni, passa al trattamento. Dopo il trattamento, torna disponibile. Se è disponibile, ma non è necessario che intervenga, può passare a occupato. Se già occupato, può restare tale o tornare disponibile.

3.1.1 Verifica delle proprietà formali

Di seguito vengono riportate le principali proprietà formali, espresse in logica CTL ed LTL, utilizzate per analizzare e verificare il corretto comportamento del sistema.

1. Se il medico è disponibile durante una situazione di allarme e il contatore dell'allarme non ha ancora raggiunto il valore massimo, allora il trattamento verrà inevitabilmente avviato prima che si verifichi un evento critico:

```
CTLSPEC AG ((paziente = allarme & medico = disponibile &
               tempo_allarme < 3)
             -> AF (paziente = trattamento & !evento_critico))
```

```
LTLSPEC G ((paziente = allarme & medico = disponibile &
               tempo_allarme < 3)
             -> F (paziente = trattamento & !evento_critico))
```

In alternativa, l'uso dell'operatore X al posto di F consente di verificare che il trattamento inizi esattamente al passo successivo rispetto all'attivazione dell'allarme.

2. Nel caso in cui il medico non sia disponibile, ma l'infermiere abbia già ricevuto istruzioni e l'allarme sia attivo da meno di 3 step, allora il trattamento sarà comunque possibile, e si potrà raggiungere uno stato in cui esso è attivo e l'evento critico non si verifica:

```
CTLSPEC AG (paziente = allarme & medico != disponibile &
               infermiere = istruzioni & tempo_allarme < 3
             -> EF (paziente = trattamento & !evento_critico))
```

Questa proprietà non è esprimibile in logica LTL, poiché LTL non consente di esprimere l'esistenza di un percorso futuro in cui una certa condizione si verifica, come invece consente l'operatore EF in CTL.

Una versione più stringente della proprietà precedente richiede che il trattamento sia inevitabilmente raggiunto in ogni possibile evoluzione del sistema:

```
CTLSPEC AG (paziente = allarme & medico != disponibile &
               infermiere = istruzioni & tempo_allarme < 3
             -> AF (paziente = trattamento & !evento_critico))
```

```
LTLSPEC G (paziente = allarme & medico != disponibile &
               infermiere = istruzioni & tempo_allarme < 3
             -> F (paziente = trattamento & !evento_critico))
```

In alternativa, l'uso dell'operatore X al posto di F consente di verificare che il trattamento venga avviato immediatamente al passo successivo, nel caso in cui l'infermiere sia pronto a intervenire.

3. Se nessuna azione viene intrapresa entro tre intervalli successivi all'attivazione dell'allarme, l'evento critico è inevitabile:

```
CTLSPEC AG ((paziente = allarme & tempo_allarme >= 2 &
             medico = occupato & infermiere != istruzioni &
             infermiere != trattamento)
            -> AF evento_critico)
```

```
LTLSPEC G ((paziente = allarme & tempo_allarme >= 2 &
             medico = occupato & infermiere != istruzioni &
             infermiere != trattamento)
            -> F evento_critico)
```

L'uso dell'operatore X al posto di F consente di verificare che, se nessun operatore avvia un trattamento entro tre intervalli di tempo, l'evento critico si manifesta esattamente al quarto passo temporale.

Infine è possibile verificare che se durante lo stato di allarme il medico e l'infermiere non trattano il paziente, l'evento critico sarà inevitabilmente raggiunto:

```
LTLSPEC G ((paziente = allarme & G(medico != trattamento &
             infermiere != trattamento))
            -> F evento_critico)
```

Questa proprietà sfrutta la logica LTL per esprimere una persistenza nel tempo: se nessun operatore diventa disponibile, e il sistema resta bloccato in tale condizione, allora l'evento critico si verificherà con certezza.

La verifica formale eseguita tramite model checking con NuSMV ha confermato la correttezza comportamentale del modello rispetto ai requisiti critici.

3.2 Modello multiprocesso

Infine, è stata implementata una variante multiprocesso del modello, con l'obiettivo di rappresentare in modo più realistico e dettagliato la gestione contemporanea di più pazienti. In questa versione, ogni paziente è modellato come un processo indipendente.

La verifica è stata suddivisa in due moduli: un modulo `main`, che verrà descritto nel paragrafo successivo, e un modulo `paziente`, la cui descrizione è omessa poiché analogo a quello impiegato nella versione precedente; la logica di evoluzione degli stati rimane infatti sostanzialmente invariata.

3.2.1 Modulo main

Il modulo `main` definisce le variabili di stato che modellano la disponibilità degli operatori (medico e infermiere), i processi associati ai pazienti e una variabile che indica quale paziente si trova da più tempo nello stato di allarme.

```
MODULE main
VAR
  medico : {0,1,2,3};
  infermiere : {0,1,2,3};
  p1 : paziente(1,medico,infermiere);
  p2 : paziente(2,medico,infermiere);
  p3 : paziente(3,medico,infermiere);
  tempo_max : {0,1,2,3};
```

Codice 4: Variabili di stato modello multiprocesso

Il medico può assumere quattro stati distinti: lo stato 0 rappresenta la disponibilità, mentre i valori 1, 2 e 3 indicano che è attualmente assegnato al paziente 1, 2 o 3 rispettivamente. L'infermiere adotta la stessa logica di rappresentazione, condividendo la stessa codifica per indicare disponibilità o assegnazione a un paziente specifico. Si osservi che non è stata utilizzata la keyword *process*, al fine di mantenere un comportamento sincrono tra i vari componenti del modello. Questa scelta consente, in particolare, di garantire l'allineamento temporale del contatore `tempo_allarme`, evitando discrepanze causate dall'evoluzione asincrona dei singoli processi.

Si procede quindi con la definizione dello stato iniziale del sistema: entrambi gli operatori (medico e infermiere) sono inizialmente disponibili, mentre la variabile `tempo_max` viene impostata in modo da non assegnare priorità a nessuno dei tre pazienti.

```
ASSIGN
  init(medico) := 0;
  init(infermiere) := 0;
  init(tempo_max) := 0;
```

Codice 5: Stato iniziale modello multiprocesso

Successivamente, vengono definite le regole di transizione per ciascuna variabile di stato:

ASSIGN

```
-- Evoluzione del tempo_max
next(tempo_max) := case
    p1.tempo_allarme > 0 & p1.tempo_allarme >= p2.
        tempo_allarme & p1.tempo_allarme >= p3.
        tempo_allarme & medico != 1 & infermiere != 1 : 1;
    p2.tempo_allarme > 0 & p2.tempo_allarme > p1.
        tempo_allarme & p2.tempo_allarme >= p3.
        tempo_allarme & medico != 2 & infermiere != 2 : 2;
    p3.tempo_allarme > 0 & p3.tempo_allarme > p1.
        tempo_allarme & p3.tempo_allarme > p2.tempo_allarme
        & medico != 3 & infermiere != 3: 3;

    tempo_max = 1 & p1.stato != allarme : 0;
    tempo_max = 2 & p2.stato != allarme : 0;
    tempo_max = 3 & p3.stato != allarme : 0;
    TRUE : tempo_max;
esac;

-- Evoluzione del medico
next(medico) := case
    p1.stato = allarme & medico = 0 & tempo_max = 1 &
        infermiere != 1: 1;
    p2.stato = allarme & medico = 0 & tempo_max = 2 &
        infermiere != 2: 2;
    p3.stato = allarme & medico = 0 & tempo_max = 3 &
        infermiere != 3: 3;

    p1.stato = allarme & medico = 0 & infermiere != 1: 1;
    p2.stato = allarme & medico = 0 & infermiere != 2: 2;
    p3.stato = allarme & medico = 0 & infermiere != 3: 3;

    p1.stato = trattamento & medico = 1 : 0;
    p2.stato = trattamento & medico = 2 : 0;
    p3.stato = trattamento & medico = 3 : 0;
    TRUE : medico;
esac;

-- Evoluzione dell'infermiere
next(infermiere) := case
    p1.stato = allarme & infermiere = 0 & !(next(medico =
        1)| medico = 1): 1;
    p2.stato = allarme & infermiere = 0 & !(next(medico =
        2)| medico = 2): 2;
    p3.stato = allarme & infermiere = 0 & !(next(medico =
        3)| medico = 3): 3;
```



```
p1.stato = trattamento & infermiere = 1 : 0;  
p2.stato = trattamento & infermiere = 2 : 0;  
p3.stato = trattamento & infermiere = 3 : 0;  
TRUE : infermiere;  
esac;
```

Codice 6: Transizioni di stato modello multiprocesso

In particolare:

- ogni ramo della definizione di `tempo_max` verifica se un determinato paziente presenta il valore di `tempo_allarme` più elevato rispetto agli altri. In tal caso, la variabile viene aggiornata con l'identificativo di quel paziente; in caso contrario, mantiene il valore corrente.
- se il `medico` è disponibile e il paziente con priorità massima non è già assegnato all'infermiere, allora viene assegnato a quel paziente. In assenza di una priorità definita, il medico viene assegnato a un paziente qualsiasi con allarme attivo. Se ha completato un trattamento, torna disponibile; altrimenti, mantiene lo stato corrente.
- l'infermiere interviene su un paziente in allarme solo se il medico non è attualmente assegnato (né verrà assegnato) a quel paziente. Al termine del trattamento torna disponibile; in caso contrario, mantiene lo stato corrente.

3.2.2 Verifica delle proprietà formali

Le proprietà verificate per questo modello coincidono con quelle definite per la versione precedente, con la differenza che, in questa variante, ciascuna proprietà viene controllata individualmente per ogni paziente. Di seguito si riporta un esempio relativo alla prima proprietà:

```
CTLSPEC AG ((p1.stato = allarme & medico = 0 &
               p2.stato = osservazione &
               p3.stato = osservazione &
               p1.tempo_allarme < 3)
  -> AF (p1.stato = trattamento & !p1.evento_critico))
```

```
CTLSPEC AG ((p2.stato = allarme & medico = 0 &
               p1.stato = osservazione &
               p3.stato = osservazione &
               p2.tempo_allarme < 3)
  -> AF (p2.stato = trattamento & !p2.evento_critico))
```

```
CTLSPEC AG ((p3.stato = allarme & medico = 0 &
               p1.stato = osservazione &
               p2.stato = osservazione &
               p3.tempo_allarme < 3)
  -> AF (p3.stato = trattamento & !p3.evento_critico))
```

```
LTLSPEC G ((p1.stato = allarme & medico = 0 &
               p2.stato = osservazione &
               p3.stato = osservazione &
               p1.tempo_allarme < 3)
  -> F (p1.stato = trattamento & !p1.evento_critico))
```

```
LTLSPEC G ((p2.stato = allarme & medico = 0 &
               p1.stato = osservazione &
               p3.stato = osservazione &
               p2.tempo_allarme < 3)
  -> F (p2.stato = trattamento & !p2.evento_critico))
```

```
LTLSPEC G ((p3.stato = allarme & medico = 0 &
               p2.stato = osservazione &
               p1.stato = osservazione &
               p3.tempo_allarme < 3)
  -> F (p3.stato = trattamento & !p3.evento_critico))
```

Riferimenti

- [1] J. Bonet et al. *PIPE v2.5: a Petri Net Tool for Performance Modeling*. Accesso: luglio 2025. Disponibile su <https://www.doc.ic.ac.uk/~wjk/publications/bonet-llado-knottenbelt-puijaner-clei-2007.pdf>. 2007.
- [2] Alessandro Cimatti, E. Clarke e Fausto Giunchiglia. “NUSMV: a new Symbolic Model Verifier”. In: (apr. 1999).
- [3] *Materiale del corso di Safety Critical Systems*. Universtà degli Studi di Napoli Federico II, 2025.